Hiwot Medical Technologies: AI Development Progress & Training Report

Project: Hiwot Smart Swaddle & Connected Neonatal Mesh Pod

Author: Eyuel Wondafrash(AI Lead) & The Hiwot Engineering Team (INSA Summer Camp 2026)

Date: August 29, 2026

Current Overall Completion Status: 45%

1. Executive Summary & Current Project Status (45% Complete)

This technical progress report outlines the current development milestone of the artificial intelligence subsystem for the Hiwot Smart Swaddle & Connected Neonatal Mesh Pod. As the AI Lead, I am reporting that we have successfully achieved 45% overall project completion. Our engineering focus is centered on bridging rigorous clinical physiological data from PhysioNet with a lightweight, secure backend machine learning architecture engineered to protect preterm infants against neonatal apnea and respiratory distress in local healthcare facilities.

2. Comprehensive System Data Flow & AI Architecture

To provide a complete operational overview, the telemetry and data flow pipeline from the bedside neonate to the central nursing station operates across four sequential stages, explicitly highlighting where system intelligence resides:

Sensing Layer (Bedside): The piezoelectric respiration sensor embedded within the smart swaddle continuously detects minute chest-wall displacements and thoracic expansions caused by the infant's breathing rhythm.

Microcontroller Telemetry Bridge (ESP32): The analog electrical signals from the sensor are captured by the on-board microcontroller. The ESP32 formats these raw voltage values into structured JSON payloads and securely transmits them over local Wi-Fi. (Note: The ESP32 does not run any AI code; it acts purely as a low-power hardware telemetry transmitter).

Backend Intelligence Engine (Python FastAPI Server & AI Model): This is where the AI lives. The main AI code, model weights (apnea_model.h5), and FastAPI server reside securely on our dedicated backend computer/server. When the server receives the real-

time telemetry stream from the ESP32, the AI immediately analyzes the numerical array, processing the breathing windows to compute a dynamic anomaly probability score.

Clinical Action & Frontend Dashboard: If the AI determines that the anomaly score exceeds the clinical safety threshold, an immediate, high-priority audio-visual alarm is dispatched to the central web dashboard, alerting medical staff instantly.

3. Detailed Breakdown of Accomplished Work (0% to 45%)

Our team has successfully engineered and executed the following foundational milestones:

System Architecture Partitioning: Strategically separated edge duties from server computation, ensuring the ESP32 acts solely as a reliable hardware bridge while our heavy machine learning models run securely on a dedicated backend server.

Dataset Procurement & Sourcing: Successfully located, downloaded, unzipped, and structured the 1.6 GB Preterm Infant Cardio-Respiratory Signals Database (PICSDB) from PhysioNet, securing multi-hour continuous breathing waveforms recorded from premature neonates.

Development Environment Configuration: Configured Visual Studio Code on Windows, initialized an isolated Python virtual environment (venv), and successfully resolved package dependencies (tensorflow, wfdb, fastapi, uvicorn, numpy, pandas).

Baseline Scripting Architecture: Developed the foundational Python script (train_model.py) to parse .dat and .hea medical waveform records and segment continuous time-series breathing streams into structured sliding windows.

4. AI Architecture & Its Role in the Ecosystem

Our backend AI model is custom-tailored for clinical neonatal safety:

Where the AI Sits: Directly on the backend Python FastAPI server, ensuring powerful processing capabilities without draining battery power on the bedside ESP32.

What the AI Will Do: It ingests continuous sliding windows of infant respiration data, utilizes a 1D Temporal Convolutional Network (1D-CNN) to extract temporal waveform patterns, and outputs an anomaly probability score ranging from 0.0 (normal rhythmic breathing) to 1.0 (severe apnea/distress).

Clinical Safety Threshold: If the AI detects an anomaly score exceeding 0.85 for more than 15 consecutive seconds, the system automatically flags an emergency alert on the

central nursing station web dashboard.

5. Training Code Implementation (train_model.py)

Below is the core structure of our training script used to load waveform signals from the dataset and prepare them for neural network compilation, complete with detailed explanatory comments:

```
import os
import wfdb
import numpy as np
import tensorflow as tf
from tensorflow.keras import layers, models

# ==========================================
# STEP 1: DATASET PATH CONFIGURATION
# ==========================================
# This variable points to the exact folder location where your
# extracted PhysioNet dataset is stored in your workspace.
data_dir = "./preterm-infant-cardio-respiratory-signals-database-1.0.0"

# ==========================================
# STEP 2: LOADING CLINICAL WAVEFORM DATA
# ==========================================
# Using the wfdb library to read the physical signal (.dat) and header (.hea) files
# for the sample infant respiration record.
record_path = os.path.join(data_dir, "infant1_resp")
record = wfdb.rdrecord(record_path)

# Extract the numerical voltage/respiration amplitude array from the record object
respiration_signal = record.p_signal
```

```
print("Successfully loaded signal shape:", respiration_signal.shape)


# ========================================
# STEP 3: TIME-SERIES WINDOWING & CHUNKING
# ========================================
# Neural networks require fixed-length inputs. This function slices the continuous
# breathing stream into uniform chunks of 150 data points (windows) for the AI to parse.
def create_windows(signal_data, window_size=150):
    windows = []
    for i in range(0, len(signal_data) - window_size, window_size):
        window = signal_data[i:i + window_size]
        windows.append(window)
    return np.array(windows)

# Generate training feature matrix from the raw signal
X_train = create_windows(respiration_signal)

# Assign placeholder baseline labels (0 for normal breathing patterns)
y_train = np.zeros(len(X_train))

# ========================================
# STEP 4: BUILDING THE 1D-CNN NEURAL NETWORK
# ========================================
# A lightweight 1D Convolutional Neural Network optimized for low-latency
# medical time-series anomaly detection on server backends.
model = models.Sequential([
    # Input layer matching our 150-step sliding window shape
    layers.Input(shape=(150, X_train.shape[2] if len(X_train.shape) > 2 else 1)),
```

```
    # First Convolutional layer with ReLU activation to extract local waveform patterns
    layers.Conv1D(16, kernel_size=3, activation='relu'),
    layers.MaxPooling1D(pool_size=2),

    # Second Convolutional layer to capture deeper temporal features
    layers.Conv1D(32, kernel_size=3, activation='relu'),
    layers.GlobalAveragePooling1D(),

    # Dense hidden layers for classification reasoning
    layers.Dense(16, activation='relu'),

    # Output layer with Sigmoid activation function
    # (outputs a probability score between 0.0 for normal and 1.0 for apnea anomaly)
    layers.Dense(1, activation='sigmoid')
])

# ==========================================
# STEP 5: COMPILING & SAVING THE MODEL
# ==========================================
# Compile the model using the Adam optimizer and binary crossentropy loss metric
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

print("Model architecture compiled successfully and ready for training execution!")
```

6. Future Planning & Action Roadmap (45% to 100%)

To advance the project from our current 45% milestone to full prototype deployment, our engineering timeline is structured as follows:

Full Model Training & Serialization (Milestone -> 60%): Execute the complete training loop across all infant records in the PhysioNet dataset, tune hyperparameters, and export the trained weights as a production file (apnea_model.h5).

FastAPI Backend Server Integration (Milestone -> 75%): Finalize main.py so the backend automatically loads apnea_model.h5 into memory upon startup and processes incoming JSON POST requests securely.

ESP32 Hardware Telemetry Link (Milestone -> 90%): Program the ESP32 Wi-Fi module to stream real-time sensor array data directly to our FastAPI backend endpoint.

Dashboard UI & End-to-End Testing (Milestone -> 100%): Connect WebSockets to render live telemetry curves and instantaneous audio-visual alerts on the nursing station user interface.

image:

